

FasiriNET

Software Documentation

A Web-Based Platform for Curating Speech Datasets in Low-Resource African Languages Toward Automatic Speech Recognition

Braxton Mandara

DSAIL Research Group

Dedan Kimathi University of Technology

Nyeri, Kenya

Version 1.0 — April 2026

Document Control

Field	Value
Title	FasiriNET — Software Documentation
Version	1.0
Date	23 April 2026
Author	Braxton Mandara
Affiliation	DSAIL, Dedan Kimathi University of Technology
Status	Released
Audience	Engineers, researchers, reviewers, contributors
Related artefacts	README.md, source repository, Supabase migrations

Table of Contents

(Auto-generated chapter listing — refer to chapter headings throughout the document.)

Front matter — Document Control, List of Figures, List of Tables, Abbreviations

1. Introduction
2. Literature Review
3. Requirements Specification
4. System Design and Architecture
5. Database Design
6. Security and Access Control
7. Functional Modules
8. User Experience and Workflows
9. Implementation Details
10. Quality Assurance and Testing
11. Deployment and Operations
12. Discussion, Limitations, and Future Work
13. Conclusion

References

Appendix A — Glossary

Appendix B — Edge Function Specifications

Appendix C — Sample Dataset Export Schema

Appendix D — Figure Index and Source Hints

List of Figures

(Auto-listed below; see Appendix D for source hints.)

- 3.1 — Use Case Diagram
- 4.1 — System Context Diagram
- 4.2 — Container Diagram
- 4.3 — UML Component Diagram
- 4.4 — Deployment Diagram
- 5.1 — Entity-Relationship Diagram
- 7.1 — Voice Recording Sequence Diagram
- 7.2 — Transcription Claim Sequence Diagram
- 7.3 — Offline Recovery Flow
- 7.4 — Recording Lifecycle State Machine
- 8.1 — Volunteer User Journey Map
- 8.2 — Transcriber User Journey Map
- 8.3 — Administrator User Journey Map
- 8.4 — End-to-End Data Lifecycle Flowchart
- 9.1 — Module Dependency Graph

List of Tables

- 3.1 — Functional Requirements
- 3.2 — Non-Functional Requirements
- 5.1 — Data Dictionary (compact)
- 5.2 — Database Functions and RPCs
- 6.1 — RLS Policy Matrix
- 9.1 — Edge Functions
- 11.1 — Environment Variables

Abbreviations

Abbrev.	Expansion
ASR	Automatic Speech Recognition
BaaS	Backend-as-a-Service
CRUD	Create, Read, Update, Delete
GDPR	General Data Protection Regulation
IAA	Inter-Annotator Agreement
IDB	IndexedDB
JWT	JSON Web Token
NLP	Natural Language Processing
PWA	Progressive Web Application
RBAC	Role-Based Access Control
RLS	Row-Level Security
RPC	Remote Procedure Call
SPA	Single-Page Application
SSL	Self-Supervised Learning

Abbrev.	Expansion
SWEBOK	Software Engineering Body of Knowledge
TTL	Time-To-Live

1. Introduction

1.1 Background and Motivation

Speech is the primary medium of communication in most African societies, yet the languages spoken across the continent remain dramatically under-represented in modern Automatic Speech Recognition (ASR) systems. Commercial ASR engines achieve word-error rates below 5% for English while frequently exceeding 50% — or being entirely unavailable — for languages such as Kikuyu, Luo, Kalenjin, Kamba and many others. This linguistic gap reproduces structural inequities in education, healthcare access, agricultural advisory services, and digital civic participation. *FasiriNET* is a web-based platform designed to close this gap by enabling community-driven curation of high-quality speech-and-text corpora suitable for training open ASR models for low-resource African languages.

1.2 Problem Statement

Existing speech corpora rarely include African languages, and where they do, the recordings are typically read-speech extracted from religious or news domains, missing the spontaneous, conversational and domain-specific speech needed for real-world deployment in clinics, classrooms and farms. Crowd-sourcing platforms such as Mozilla Common Voice provide a partial blueprint, but their question banks, consent workflows, transcription pipelines and dataset-export tooling are not tailored to the realities of African data collection: intermittent connectivity, mobile-first usage, multi-dialect contributors, and the necessity of explicit, verifiable, withdrawable consent. *FasiriNET* addresses this gap with an opinionated, end-to-end pipeline from prompted recording to transcribed, exportable ASR data.

1.3 Aims and Objectives

The overarching aim of this work is to design and implement a production-grade platform that allows African-language communities to collect, transcribe, verify and export speech datasets suitable for training ASR models. Specific objectives:

- Provide a mobile-first recording experience that works on low-end devices and intermittent networks, with automatic recovery of un-uploaded recordings.
- Capture verifiable consent before any audio is recorded, and persist that consent across sessions for full auditability.
- Implement an atomic transcription-claim workflow so that multiple transcribers cannot accidentally duplicate work.
- Enforce strict role-based access control between Volunteers, Transcribers, and Administrators using a database-backed roles table and Row-Level Security.
- Produce machine-readable dataset exports (audio + transcription + metadata) ready to feed into wav2vec 2.0, Whisper or MMS fine-tuning pipelines.

1.4 Scope and Limitations

Within scope: the *FasiriNET* web client (React 18 + Vite SPA), its Supabase-backed BaaS layer (PostgreSQL, Auth, Storage, Edge Functions), the transcription workflow, the administrative QA dashboard, and the dataset-export tooling. Out of scope for this version: training of ASR models themselves, native iOS/Android wrappers, and real-time speech-to-text inference. The platform currently focuses on Kikuyu as the first target language; the architecture is language-agnostic.

1.5 Document Conventions and Structure

This document follows IEEE/SWEBOK-style software-documentation conventions. Chapters are numbered (1, 2, ...); sub-sections use dotted notation (1.1, 1.2). Inline citations appear as [n] and resolve against the numbered Reference list. Figures use boxed placeholders with figure number, caption and source hint — see Appendix D for an index. Source code references use monospace.

2. Literature Review

2.1 Automatic Speech Recognition: An Overview

Modern ASR systems map a sequence of acoustic frames to a sequence of orthographic tokens. Hidden-Markov-Model / Gaussian-Mixture-Model architectures dominated the field until the 2010s, when end-to-end neural approaches — connectionist temporal classification, attention-based encoder-decoders, and RNN-T — overtook them on every major benchmark [1], [2]. Today, the dominant paradigm is large pre-trained self-supervised acoustic models fine-tuned with a small amount of labelled speech.

2.2 The Low-Resource Problem in African NLP

Joshi et al. [3] demonstrated that the world's languages cluster sharply by NLP resource level: a handful of high-resource languages (English, Mandarin, Spanish) dominate, while the long tail — including most African languages — has near-zero labelled data. The Masakhane initiative [4], [5] has shown that participatory, community-led research can produce credible NLP resources for African languages, and that data scarcity, not modelling, is the binding constraint.

2.3 Existing Speech Corpora and Crowdsourcing Platforms

Mozilla Common Voice [6] pioneered web-based crowdsourced speech collection and now covers 100+ languages, but African languages remain a small minority of total hours. Multilingual LibriSpeech [7] and FLEURS [8] provide read-speech baselines but are heavily skewed to high-resource languages. Domain-specific spontaneous African speech, especially in health and education, remains largely uncollected.

2.4 Self-Supervised Learning for ASR

wav2vec 2.0 [2] introduced contrastive self-supervised pre-training on raw audio, enabling competitive ASR with as little as ten minutes of labelled data per language. XLS-R [9] scaled this to 128 languages, and Meta's Massively Multilingual Speech (MMS) project [10] extended coverage to over 1,100 languages including many African ones. OpenAI's Whisper [11] demonstrated that weakly-supervised internet-scale training also yields strong multilingual ASR. All of these models depend critically on the existence of clean, well-transcribed evaluation and fine-tuning data — exactly the gap FasiriNET addresses.

2.5 Health-Domain Speech Technology in Low-Resource Settings

The WHO Global Strategy on Digital Health 2020-2025 [12] identifies voice interfaces as a key enabler for last-mile health information delivery. Targeted health-domain ASR is therefore a high-impact application of low-resource speech corpora, motivating FasiriNET's prompt bank to include health questions alongside general-domain ones.

2.6 Ethics, Consent and Data Sovereignty

Bender and Friedman's data statements [13] argue that every NLP dataset should ship with explicit metadata about its speakers, collection conditions and intended use. Nekoto et al. [4] further argue for data sovereignty: African-language data should remain accessible to and controlled by the communities that produced it. FasiriNET operationalises both of these recommendations through database-backed consent and open dataset exports.

2.7 Research Gap Addressed by FasiriNET

Three gaps motivate this work. First, no widely-deployed crowdsourcing platform is purpose-built for African low-resource speech with first-class support for spontaneous responses to prompts. Second, most platforms treat transcription as an afterthought, lacking atomic claim-and-lock workflows. Third, consent and audit are typically session-scoped rather than persistent. FasiriNET closes these gaps with an integrated, role-based, mobile-first pipeline.

3. Requirements Specification

3.1 Stakeholders and User Roles

FasiriNET defines three primary roles, each with distinct goals and access scope:

- **Volunteer** — community member who provides spoken responses to curated prompts. Has read access to questions and write access only to their own recordings and progress.
- **Transcriber** — vetted contributor (admin-approved via the `transcription_approved` profile flag) who claims and submits text transcriptions for accepted recordings.
- **Administrator** — manages users, verifies recordings, curates the question bank, monitors activity logs, and exports the dataset.

3.2 Functional Requirements

Table 3.1 — Functional Requirements

ID	Requirement	Priority
FR-01	Email/password account creation with strength enforcement	Must
FR-02	Multi-step signup capturing demographic and dialect metadata	Must
FR-03	Persistent voice-recording consent before any capture	Must
FR-04	Browse questions grouped by category	Must
FR-05	Record audio responses with 30-second cap and waveform feedback	Must
FR-06	Auto-save un-uploaded recordings to IndexedDB and offer recovery	Must
FR-07	Retire questions automatically after 3 unique-user responses	Must
FR-08	Atomically claim a random untranscribed accepted recording	Must
FR-09	Submit and edit transcriptions (max 3 edits per record)	Must
FR-10	Admin verification queue (accept / reject) of recordings	Must
FR-11	Export curated dataset as ZIP of audio + JSON/CSV metadata	Must

ID	Requirement	Priority
FR-12	Admin dashboard with charts and per-user activity	Should
FR-13	Real-time presence tracking of active users	Should
FR-14	Drive-to-Storage migration of legacy audio	Should
FR-15	PWA install prompt and offline shell	Could

3.3 Non-Functional Requirements

Table 3.2 — Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Initial dashboard load under 3s on 3G; recording UI under 1s
NFR-02	Reliability	Zero-loss recording; every captured blob recoverable for ≥24h
NFR-03	Security	RLS on every table; roles in separate table; JWT auth
NFR-04	Privacy	Verifiable consent timestamp; right-to-withdraw
NFR-05	Accessibility	WCAG 2.1 AA; keyboard nav; skip-to-content; ARIA labels
NFR-06	Mobile UX	Touch targets ≥44px; haptic feedback; pull-to-refresh
NFR-07	Offline	Service Worker shell; recording works without network
NFR-08	Auditability	Activity log of every user action with timestamp
NFR-09	Maintainability	Typed end-to-end (TypeScript + generated DB types)
NFR-10	Portability	Runs on any modern browser; PWA installable

3.4 Use-Case Summary

Fourteen primary use cases span the three roles: *Register/Login*, *Grant Consent*, *Browse Categories*, *Record Response*, *Review Own Recordings* for Volunteers; *Agree to Transcription Guidelines*, *Claim Task*, *Submit Transcription*, *Review Own Transcriptions* for Transcribers; and *View Dashboard Analytics*, *Manage Users & Roles*, *Verify Responses*, *Export Dataset*, *Monitor Activity Logs* for Administrators. The graphical use-case diagram is rendered below.



Figure 3.1 — Use Case Diagram showing the three actor roles and their interactions.

4. System Design and Architecture

4.1 Architectural Overview

FasiriNET follows a three-tier architecture: a single-page React client, a Backend-as-a-Service (BaaS) layer providing PostgreSQL, authentication, object storage and serverless edge functions, and a downstream ASR-dataset output stage. The client is fully stateless on the server; all state lives in PostgreSQL or in the user's browser (IndexedDB / localStorage) for resilience.

4.2 System Context

The system context places FasiriNET between three external actors (Volunteer, Transcriber, Administrator) and three external systems (Email provider for verification, optional Google Drive for legacy migration, and the downstream ASR training pipeline that consumes exports).

[INSERT FIGURE 4.1 HERE]

Source: Hand-drawn C4 diagram or draw.io export

Figure 4.1 — System Context Diagram (C4 Level 1).

4.3 Container View

Within the FasiriNET boundary there are four logical containers: the React SPA, the Supabase Auth service, the PostgreSQL database with RLS, and the Storage bucket holding audio blobs. Edge Functions execute Deno-based server-side logic for tasks unsuited to the client, such as signed-URL generation.

[INSERT FIGURE 4.2 HERE]

Source: Hand-drawn C4 diagram or draw.io export

Figure 4.2 — Container Diagram (C4 Level 2).

4.4 Component View

Inside the SPA, components are organised by domain: Auth (Login, Signup, ProtectedRoute, AdminRoute), Dashboard (CategoryCard, QuestionsPreviewCard, ProfileDialog), Questions (QuestionCard, ConsentModal), VoiceRecording (VoiceRecorder, MinimalRecorder, AudioWaveform), Transcription, and Admin (a tabbed administration panel). Cross-cutting infrastructure includes React Query for server state, React Router for navigation, and a set of custom hooks (useHapticFeedback, useSwipeGesture, usePullToRefresh, useUserRole).

[INSERT FIGURE 4.3 HERE]

Source: README.md → "Component Architecture" Mermaid block

Figure 4.3 — UML Component Diagram of the SPA and BaaS layers.

4.5 Technology Stack Rationale

React 18 + Vite was chosen for fast HMR and small production bundles. TypeScript provides end-to-end type safety against Supabase-generated database types. Tailwind CSS + shadcn/ui give a consistent, themeable design system with minimal runtime cost. Supabase delivers Postgres with RLS, JWT auth, storage and edge functions in a single, managed BaaS — eliminating the need for a separate API tier.

4.6 Deployment Topology

The SPA is built statically and served from Vercel's CDN. The backend runs on Lovable Cloud's managed Supabase. Edge functions are deployed automatically on push. There is no long-lived server to operate.

[INSERT FIGURE 4.4 HERE]

Source: Hand-drawn deployment diagram

Figure 4.4 — Deployment Diagram showing CDN, BaaS and edge runtime.

5. Database Design

5.1 Conceptual Data Model

Entities centre on the **profile** (the human contributor), the **question** (prompt to be answered), the **voice_response** (an audio recording answering a question), and the **transcription** (the textual rendering of a voice_response). Around these revolve **category** (group of questions), **user_progress** (per-user, per-category counters), **user_role** (RBAC), **user_activity_log** (audit), and **transcription_lock** (atomic claim TTL).

5.2 Logical Schema

[INSERT FIGURE 5.1 HERE]

Source: README.md → "Entity-Relationship Diagram" Mermaid block

Figure 5.1 — Entity-Relationship Diagram of the nine-table schema.

5.3 Compact Data Dictionary

Table 5.1 — Data Dictionary — selected columns per entity (audit fields omitted).

Table	Column	Type	Notes
profiles	id	uuid PK	FK to auth.users (1:1)
	email, first_name, last_name, phone_number	text	Core identity
	gender, age, dialect	text/int	Demographics for dataset metadata
	account_type	text	volunteer transcriber
	verified	bool	Email verified
	transcription_approved	bool	Admin-granted transcriber privilege
	voice_recording_consent	bool	Persistent consent flag
	consent_timestamp	timestamp	When consent was last granted
categories	id, name, description	uuid/text	Question grouping
questions	id, category_id, question_text	uuid/text	Prompt body
	order_index, image_url, image_attribution	int/text	UI ordering & visuals
voice_responses	id, user_id, question_id	uuid	Identity of the recording
	audio_file_url	text	Storage path
	duration_seconds	float	Length of audio
	status	enum	pending accepted rejected
	response_type	text	audio text fallback

Table	Column	Type	Notes
transcriptions	id, voice_response_id, user_id	uuid	Authorship
	transcription_text	text	Verbatim transcription
	edit_count	int	Capped at 3
	status	text	submitted verified rejected
transcription_locks	id, voice_response_id, user_id	uuid	1:1 with response while held
	expires_at	timestamp	now() + 30 minutes
user_roles	id, user_id, role	uuid/enum	app_role: admin user
user_progress	id, user_id, category_id	uuid	Composite key
	completed_questions, total_questions	int	Per-category counters
user_activity_logs	id, user_id, action, page, details	uuid/text/json	Audit trail

5.4 Constraints and Integrity Rules

- **Unique (user_id, question_id)** on `voice_responses` prevents duplicate submissions for the same prompt by the same user.
- **Question retirement** via `get_question_unique_user_counts()` auto-hides questions once 3 distinct users have answered them.
- **Edit cap** on transcriptions enforced at the application layer (max 3) and tracked by `edit_count`.
- **Lock TTL**: `transcription_locks.expires_at` defaults to `now() + 30 minutes`; expired locks are reclaimable.

5.5 Database Functions and RPCs

Table 5.2 — Database Functions and RPCs.

Function	Purpose
<code>has_role(_user_id, _role)</code>	SECURITY DEFINER role check; the foundation of every RLS policy.
<code>claim_random_transcription(_user_id)</code>	Atomic SELECT-FOR-UPDATE + INSERT lock; returns one untranscribed accepted response.
<code>release_transcription_lock(_user_id)</code>	Releases or expires the caller's active lock.
<code>get_question_unique_user_counts()</code>	Returns per-question count of distinct contributors; powers retirement.
<code>get_question_counts()</code>	Aggregate question metrics for the admin dashboard.
<code>get_admin_chart_data()</code>	Pre-aggregated time-series JSON for admin charts.

6. Security and Access Control

6.1 Authentication

Authentication uses Supabase GoTrue with email/password only. Email verification is required before login. Password strength is enforced client-side with a strength indicator and server-side by GoTrue policy. Password reset is delivered by an edge function (`reset-password`) that issues a signed magic link. There is no anonymous sign-in and no third-party OAuth in this version.

6.2 Role-Based Access Control

Roles live in a dedicated `user_roles` table — never on `profiles` — to prevent privilege-escalation via profile updates. The `app_role` enum currently has values `admin` and `user`; transcriber privilege is layered on top via the `transcription_approved` boolean on the profile, gated by admin verification. The `has_role()` SECURITY DEFINER function lets RLS policies check role membership without recursion.

6.3 RLS Policy Matrix

Table 6.1 — Row-Level Security policy matrix (✓ = allowed under noted condition).

Table	Volunteer	Transcriber	Admin
profiles	✓ self	✓ self	✓ all
categories	✓ read	✓ read	✓ all
questions	✓ read (active)	✓ read	✓ all

Table	Volunteer	Transcriber	Admin
voice_responses	✓ self CRUD	✓ read accepted (metadata + audio URL)	✓ all
transcriptions	— none	✓ self CRUD	✓ all
transcription_locks	— none	✓ self	✓ all
user_roles	— none	— none	✓ all
user_progress	✓ self	✓ self	✓ all
user_activity_logs	✓ self insert	✓ self insert	✓ all

6.4 Privilege-Escalation Prevention

Because role assignment is gated by admin-only RLS on `user_roles`, no client-side mutation can grant elevated privilege. The `has_role` function is SECURITY DEFINER with an explicit `SET search_path = public`, defending against search-path injection attacks.

6.5 Data Privacy and Consent

Voice-recording consent is captured before the first recording and persisted in `profiles.voice_recording_consent` together with `consent_timestamp`. Consent is withdrawable; on withdrawal, the user retains the right to request deletion of their recordings. Demographic fields are optional and used only as dataset metadata.

6.6 Transport, Storage and Key Management

All traffic is TLS-terminated by the CDN. Audio blobs live in a private Storage bucket; access is mediated by RLS-equivalent storage policies. Application code uses the publishable anon key only; service-role keys are confined to edge functions and provided via server-side environment variables.

7. Functional Modules

7.1 Voice Recording Module

The recording flow begins on the Dashboard: the user selects a category, navigates to `/questions`, and is presented with a `QuestionCard`. Tapping *Record* activates the `MediaRecorder` API and renders an `AudioWaveform` visualisation. A 30-second cap is enforced. While recording, the blob is incrementally serialised to IndexedDB by `recordingStorage.ts`. On submit, the blob is uploaded to Storage; on success, `voice_responses` and `user_progress` are updated atomically and the IDB backup is cleared.

[INSERT FIGURE 7.1 HERE]

Source: *README.md* → "Recording Sequence" Mermaid block

Figure 7.1 — Voice Recording Sequence Diagram.

7.2 Transcription Module with Atomic Locking

Transcribers gated by `transcription_approved` navigate to `/transcribe` and call the `claim_random_transcription` RPC. The function opens a transaction, selects a random untranscribed accepted response with `FOR UPDATE SKIP LOCKED`, inserts a row in `transcription_locks` with TTL 30 minutes, and returns the response payload. On submit, the transcription is inserted and the lock released. Expired locks are reclaimable on the next call. Edits are capped at 3 via `edit_count`.

[INSERT FIGURE 7.2 HERE]

Source: README.md → "Transcription Sequence" Mermaid block

Figure 7.2 — Transcription Claim Sequence Diagram.

7.3 Administration and QA Module

The Admin panel is a tabbed SPA region behind `AdminRoute`. Tabs include Overview (charts), All Users / Active Users, Questions, Responses (with debounced server-side search), Transcriptions (with transcriber-approval controls), Dataset (HuggingFace-style table view + ZIP export), Activity Logs, and Drive Backfill (legacy audio reconciliation).

7.4 Dataset Curation and Export

The Dataset tab assembles accepted recordings with verified transcriptions into a downloadable ZIP archive containing the audio files plus a JSON/CSV manifest with speaker demographics, dialect, duration, and dual attribution (recorder + transcriber). This format is directly consumable by HuggingFace `datasets.load_dataset`.

7.5 Mobile-First PWA and Offline Resilience

A Service Worker caches the application shell for offline launch. Recordings made while offline are saved to IndexedDB; on reconnection, `RecoveryDialog` prompts the user to upload them. The `OfflineIndicator` banner makes connectivity state explicit. Mobile UX is enhanced by haptic feedback on key actions, swipe navigation between questions, and pull-to-refresh on lists.

[INSERT FIGURE 7.3 HERE]

Source: Hand-drawn or repo Mermaid (recovery flowchart)

Figure 7.3 — Offline Recovery Flow.

7.6 Activity Tracking and Realtime Presence

Every meaningful user action writes a row to `user_activity_logs` via the `useActivityTracker` hook. A Supabase Realtime subscription (`PresenceTracker`) tracks currently-online users and surfaces the count in the Admin Active Users tab.

7.7 Recording Lifecycle State Machine

A recording transitions through Pending → Accepted/Rejected; an Accepted recording may be Claimed for transcription, transitioning the lock through LockActive → LockExpired (reclaimable) or → TranscriptionSubmitted, which itself may be Verified or TranscriptionRejected (returning to Claimed).

[INSERT FIGURE 7.4 HERE]

Source: README.md → flat stateDiagram-v2 block

Figure 7.4 — Recording Lifecycle State Machine (response + lock + transcription).

8. User Experience and Workflows

8.1 Volunteer Journey

Discovery → Sign-up (4-step wizard collecting demographics, dialect, role) → Email verification → First login → Consent modal → Dashboard → Pick a category → Record first response → Confetti success → Continue or revisit /my-responses for a summary.

[INSERT FIGURE 8.1 HERE]

Source: Custom journey-map figure

Figure 8.1 — Volunteer User Journey Map.

8.2 Transcriber Journey

Sign-up with role=transcriber → Await admin approval (transcription_approved=true) → Read and agree to transcription guidelines (Kikuyu-specific orthography, fillers like [Pause], [uh]) → /transcribe claims a task → Listen, transcribe, submit → /my-transcriptions review.

[INSERT FIGURE 8.2 HERE]

Source: Custom journey-map figure

Figure 8.2 — Transcriber User Journey Map.

8.3 Administrator Journey

Login → Admin panel → Overview charts → Approve pending transcribers → Verify voice responses → Curate question bank → Run drive backfill if needed → Export dataset ZIP for ASR pipeline.

[INSERT FIGURE 8.3 HERE]

Source: Custom journey-map figure

Figure 8.3 — Administrator User Journey Map.

8.4 End-to-End Data Lifecycle

From prompt creation by an admin, through volunteer recording, IDB backup, upload, admin verification, transcriber claim, transcription submission, transcription verification, to inclusion in the next dataset export — a single accepted recording traverses six logical states across three roles.

[INSERT FIGURE 8.4 HERE]

Source: Custom flowchart (Mermaid graph TB)

Figure 8.4 — End-to-End Data Lifecycle Flowchart.

8.5 Accessibility and Inclusivity Guardrails

- Skip-to-content link for keyboard users.
- Skip-confirmation dialog before destructive recording skips.
- ARIA labels on icon-only buttons; visible focus rings.
- Light/dark themes with WCAG-AA contrast in both.
- Pull-to-refresh and swipe gestures only as enhancements, never sole affordances.

9. Implementation Details

9.1 Project Structure

The codebase is organised by domain under `src/`: `pages/` (route-level views), `components/` grouped by feature (Auth, Dashboard, Questions, Transcription, VoiceRecording, Admin), `hooks/` for cross-cutting behaviour, `services/` for I/O wrappers, `utils/` for pure helpers, and `integrations/supabase/` for the auto-generated client and types.

9.2 Frontend Implementation Patterns

- **Server state via React Query:** every Supabase read is a `useQuery`; every mutation is a `useMutation` with optimistic invalidation.
- **Type safety:** `src/integrations/supabase/types.ts` is generated from the live schema and imported throughout.
- **Design system:** shadcn/ui primitives styled via Tailwind semantic tokens defined in `index.css` (HSL only).
- **Routing:** React Router v6 with `ProtectedRoute` and `AdminRoute` wrappers.
- **Custom hooks:** `useUserRole`, `useActivityTracker`, `usePresenceTracking`, `useHapticFeedback`, `useSwipeGesture`, `usePullToRefresh`.

9.3 Edge Functions

Table 9.1 — Edge Functions deployed alongside the SPA.

Function	Purpose
reset-password	Issues a signed magic link for password reset via email.

Function	Purpose
upload-to-gcs	Server-side signed-URL upload helper (legacy migration target).
migrate-drive-audio	Ferries legacy Google Drive audio into the Storage bucket.

9.4 Atomic Transcription Locking — Pseudocode

```

FUNCTION claim_random_transcription(_user_id) RETURNS row
BEGIN
  release_expired_locks();
  SELECT vr.* INTO target
  FROM voice_responses vr
  LEFT JOIN transcriptions t ON t.voice_response_id = vr.id
  LEFT JOIN transcription_locks l ON l.voice_response_id = vr.id
  WHERE vr.status = 'accepted' AND t.id IS NULL AND l.id IS NULL
  ORDER BY random() LIMIT 1
  FOR UPDATE SKIP LOCKED;
  INSERT INTO transcription_locks(voice_response_id, user_id, expires_at)
  VALUES (target.id, _user_id, now() + interval '30 minutes');
  RETURN target;
END;

```

9.5 Offline-First Storage

`recordingStorage.ts` wraps `IndexedDB` with a small key/value API: every Blob is base64-encoded and stored under a synthetic key derived from the question id and a timestamp. On app startup, `RecoveryDialog` scans for un-uploaded keys and offers to retry; successful uploads delete the corresponding key.

[INSERT FIGURE 9.1 HERE]

Source: Custom diagram derived from import graph

Figure 9.1 — Module Dependency Graph (top-level packages).

10. Quality Assurance and Testing

10.1 Testing Strategy

FasiriNET currently relies on a layered manual-plus-automatic strategy: TypeScript and the generated Supabase types provide compile-time integrity; React Query ensures that UI invariants always reflect the latest server state; admin manual QA handles dataset curation. Future work includes Playwright end-to-end coverage of the recording and claim flows.

10.2 Manual QA Workflows

Admins listen to each pending recording in the Responses tab and accept or reject it. Rejected recordings are excluded from transcription claims and from the dataset export. Verified transcriptions are similarly gate-kept before export.

10.3 Question Retirement

After three unique users have answered a question, the `get_question_unique_user_counts` RPC marks it as retired, removing it from active prompts for all users. This ensures even sample-size distribution across the question bank.

10.4 Edit-Count Gating

Each transcription tracks `edit_count`, capped at 3. After three edits the record is locked from further modification; subsequent corrections require admin intervention. This balances quality with anti-abuse.

10.5 Inter-Annotator Agreement (Future Work)

Multiple-transcriber-per-recording mode is planned for v2, with Cohen's κ and character-error-rate metrics surfaced in the admin dashboard.

11. Deployment and Operations

11.1 Hosting

The SPA is built with `vite build` and served from Vercel's CDN. The backend is provisioned as a managed Supabase project (Lovable Cloud). No long-running servers exist.

11.2 Migrations Workflow

Database changes are forward-only SQL files in `supabase/migrations/`, applied automatically on deploy. Generated types are refreshed on every schema change.

11.3 Environment Variables

Table 11.1 — Environment variables required at build and runtime.

Variable	Scope	Purpose
VITE_SUPABASE_URL	Build	BaaS endpoint base URL
VITE_SUPABASE_PUBLISHABLE_KEY	Build	Anon JWT key for browser client
VITE_SUPABASE_PROJECT_ID	Build	Project identifier
SUPABASE_SERVICE_ROLE_KEY	Edge	Privileged key, edge functions only
GOOGLE_DRIVE_FOLDER_ID	Edge	Source folder for audio migration

11.4 Backup and Recovery

Supabase performs nightly snapshots of PostgreSQL and Storage. Point-in-time recovery is available within the retention window. Client-side IndexedDB plus `RecoveryDialog` protects against data loss during interrupted uploads.

11.5 Monitoring

`user_activity_logs` provides an in-database audit trail. Edge function logs and PostgreSQL logs are accessible via the Supabase dashboard. The Active Users tab leverages Realtime presence to show live concurrency.

12. Discussion, Limitations, and Future Work

12.1 Achievements

FasiriNET delivers a production-grade end-to-end pipeline from prompted recording to exportable ASR-ready dataset, with first-class support for consent, mobile UX, offline resilience and atomic transcription claiming — features absent from comparable open platforms.

12.2 Current Limitations

- Single primary target language at launch (Kikuyu); language-agnostic but untested at scale.
- No automated end-to-end tests yet; reliance on manual QA.
- Admin moderation does not yet support multi-reviewer consensus.
- Dataset export is on-demand rather than scheduled and incremental.

12.3 Roadmap

- Multi-language deployment with per-language admin teams.
- Semi-automated transcription using XLS-R/MMS pre-fills the transcriber edits.
- ASR benchmarking inside the Admin panel (CER/WER on held-out splits).
- Inter-annotator agreement metrics (Cohen's κ , character-level alignment).
- Native mobile shell via Capacitor for richer offline integration.

12.4 Sustainability and Community Model

The platform is open-source and designed to be operated by community organisations. Long-term sustainability depends on a federated model in which language communities host their own instances, with shared dataset standards enabling cross-instance pooling.

13. Conclusion

FasiriNET demonstrates that production-quality crowd-sourced speech-data collection for low-resource African languages is achievable on a modern BaaS stack. By combining a mobile-first React SPA, PostgreSQL with strict RLS, atomic transcription locking, persistent consent, and offline resilience, the platform removes the engineering friction that has historically limited African ASR data collection to one-off academic efforts. The architecture is intentionally

language-agnostic and intentionally aligned with the data-sovereignty principles of the Masakhane community, providing a credible foundation for the next generation of inclusive speech technology in Africa.

References

- [1] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, 2006, pp. 369–376.
- [2] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Proc. NeurIPS*, vol. 33, 2020, pp. 12449–12460.
- [3] P. Joshi, S. Santy, A. Budhiraja, K. Bali, and M. Choudhury, "The state and fate of linguistic diversity and inclusion in the NLP world," in *Proc. ACL*, 2020, pp. 6282–6293.
- [4] W. Nekoto et al., "Participatory research for low-resourced machine translation: A case study in African languages," in *Findings of EMNLP*, 2020, pp. 2144–2160.
- [5] D. I. Adelani et al., "MasakhaNER 2.0: Africa-centric transfer learning for named entity recognition," in *Proc. EMNLP*, 2022, pp. 4488–4508.
- [6] R. Ardila et al., "Common Voice: A massively-multilingual speech corpus," in *Proc. LREC*, 2020, pp. 4218–4222.
- [7] V. Pratap, Q. Xu, A. Sriram, G. Synnaeve, and R. Collobert, "MLS: A large-scale multilingual dataset for speech research," in *Proc. Interspeech*, 2020, pp. 2757–2761.
- [8] A. Conneau et al., "FLEURS: Few-shot learning evaluation of universal representations of speech," in *Proc. SLT*, 2022, pp. 798–805.
- [9] A. Babu et al., "XLS-R: Self-supervised cross-lingual speech representation learning at scale," in *Proc. Interspeech*, 2022, pp. 2278–2282.
- [10] V. Pratap et al., "Scaling speech technology to 1,000+ languages," *Journal of Machine Learning Research*, vol. 25, 2024.
- [11] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023, pp. 28492–28518.
- [12] World Health Organization, *Global Strategy on Digital Health 2020–2025*. Geneva: WHO, 2021.
- [13] E. M. Bender and B. Friedman, "Data statements for natural language processing: Toward mitigating system bias and enabling better science," *Transactions of the ACL*, vol. 6, pp. 587–604, 2018.
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [15] D. Amodei et al., "Deep Speech 2: End-to-end speech recognition in English and Mandarin," in *Proc. ICML*, 2016, pp. 173–182.
- [16] K. Ochieng, P. Wanjawa, and B. Indede, "A survey of speech recognition for low-resource African languages," *ACM Computing Surveys*, 2024 (in press).
- [17] T. Hernandez Mena, A. Black, and L. F. D'Haro, "Design considerations for crowdsourced speech corpora in under-resourced languages," in *Proc. SLTU*, 2018, pp. 16–20.
- [18] S. Watanabe et al., "ESPnet: End-to-end speech processing toolkit," in *Proc. Interspeech*, 2018, pp. 2207–2211.
- [19] Mozilla Foundation, *Common Voice Dataset v17.0*. [Online]. Available: <https://commonvoice.mozilla.org>, 2024.
- [20] HuggingFace, *Datasets: A community library for natural language processing*, 2023. [Online]. Available: <https://huggingface.co/docs/datasets>.

Appendix A — Glossary

Term	Definition
Accepted recording	A voice_response with status=accepted, eligible for transcription.
Atomic claim	Indivisible SELECT-FOR-UPDATE+INSERT-lock that prevents two transcribers picking the same task.
Consent timestamp	Server-recorded time at which a user granted voice-recording consent.
Edit cap	The maximum number of times (3) a transcription may be revised.
Lock TTL	Time-to-live of a transcription lock (30 minutes); past TTL the task is reclaimable.
Question retirement	Automatic deactivation of a question once 3 unique users have answered it.
Verified transcription	A transcription with status=verified, eligible for inclusion in the dataset export.

Appendix B — Edge Function Specifications

B.1 reset-password

Method: POST. Body: { email }. Behaviour: looks up the user, generates a signed magic-link, dispatches via the configured email provider. Returns 200 even when the email does not exist (anti-enumeration).

B.2 upload-to-gcs

Method: POST. Body: { filename, contentType }. Returns a signed PUT URL. Used by legacy migration; current uploads go directly to Supabase Storage via the SDK.

B.3 migrate-drive-audio

Method: POST. Body: { folderId, dryRun }. Iterates the source Google Drive folder, downloads each audio file, uploads it to the voice-recordings bucket, and updates the matching voice_responses.audio_file_url.

Appendix C — Sample Dataset Export Schema

```
{
  "id": "<uuid>",
  "audio_path": "audio/<uuid>.webm",
  "duration_seconds": 24.7,
  "transcription": "<verbatim text>",
  "language": "kik",
  "dialect": "<dialect code>",
  "speaker": { "gender": "f", "age_band": "25-34" },
  "category": "health",
  "question": "<prompt text>",
  "recorder_id": "<hashed uuid>",
  "transcriber_id": "<hashed uuid>",
  "recorded_at": "2026-04-01T10:23:55Z",
  "verified": true
}
```

The companion CSV is a flat projection of the same fields, suitable for `datasets.load_dataset('csv', data_files=...)`.

Appendix D — Figure Index and Source Hints

Figure	Caption	Source hint
3.1	Use Case Diagram	README.md → "Use Case Diagram" Mermaid block
4.1	System Context Diagram	Hand-drawn C4 Level 1
4.2	Container Diagram	Hand-drawn C4 Level 2
4.3	UML Component Diagram	README.md → "Component Architecture" Mermaid
4.4	Deployment Diagram	Hand-drawn
5.1	ER Diagram	README.md → "Entity-Relationship Diagram" Mermaid
7.1	Recording Sequence	README.md → "Recording Sequence" Mermaid
7.2	Transcription Claim Sequence	README.md → "Transcription Sequence" Mermaid
7.3	Offline Recovery Flow	Custom flowchart
7.4	Lifecycle State Machine	README.md → flat stateDiagram-v2
8.1	Volunteer Journey Map	Custom journey-map
8.2	Transcriber Journey Map	Custom journey-map
8.3	Admin Journey Map	Custom journey-map
8.4	Data Lifecycle Flowchart	Custom flowchart
9.1	Module Dependency Graph	Derived from import graph